Intants — Production Expansion Plan

> ***Status:*** *Planning document (v1, 2026-06-17)* ***Owner:*** *Intants engineering* ***Scope:*** *Evolving the current 1:1 voice-avatar interviewer into a production platform with candidate video, gaze-based proctoring, bulk hiring, multiple intake scenarios, and production-grade security. This document describes* ***what we will build and why****. It does not contain final code. Each phase links back to the real services it touches so the work is traceable. Read alongside* `HLD.md`*,* `LLD.md`*, and* `Final_stack.md`*.*

---

1. Where we are today (baseline)

The platform is a **1:1 voice-first AI interview system** and it works end-to-end:

Capability	Where it lives	State
Real-time avatar interview (Tavus over LiveKit)	`services/interview_core` (`app/worker/interview_worker.py`)	■ Working
Voice pipeline (Sarvam STT/TTS, Silero VAD)	`interview_core/app/speech/`	■ Working
Interviewer brain / question policy	`interview_core/app/worker/interview_worker.py` (`_interviewer_instructions`)	■ Working
Resume-grounded questions	resume text injected into the interviewer prompt	■ Working
Auth + users + consent (pluggable)	`services/data_gateway`	■ Working
Scoring + PDF scorecard	`services/feedback_billing`	■ Working (needs Gemini key)
Admin dashboard + analytics	`services/admin_ops` + `web/src/pages/admin/`	■ Working

Two things the current product does NOT have, and this plan adds:

- **No candidate video / camera.** The candidate is audio-only; only the avatar has a video track. There is no webcam capture, no candidate-side video, no true two-way "face-to-face" feel.
- **No proctoring / integrity signals.** There is no detection of looking away, absence, multiple faces, or tab-switching. Nothing prevents or flags malpractice.

Everything below is **additive** and env-swappable, consistent with the two-tier strategy in `Final_stack.md`.

---

2. Goals of this expansion

- **Candidate video + true 1:1 interaction** — the candidate's webcam is live in the session (shown to themselves, captured for proctoring), alongside the avatar. The session feels face-to-face.
- **Gaze / eye-movement proctoring** — detect when the candidate looks away from the screen, leaves the frame, has multiple faces present, or switches tabs. Produce an **integrity score** for human review.
- **Bulk hiring at scale** — one or many companies running thousands of interviews in a hiring drive, with invites, batches, and a recruiter console.
- **Multiple intake scenarios** — off-campus single applicant, resume-driven screening, and fully admin-managed campaigns — all on one engine.

- **Production security** — the controls required before this touches real

candidate PII and biometric data at scale (DPDP Act 2023 + RFP NFRs).

---

3. A note on terminology — "eye tracking" vs what is actually feasible

This matters for credibility, especially with government buyers.

- **True retina / iris tracking** (knowing the exact pixel a person is looking at)

requires **infrared hardware** (e.g. Tobii). It is **not possible with a normal webcam.** We will not claim it.

- **What is feasible and is what real proctoring products ship:** webcam-based

gaze direction + head pose estimation (approximate "is the candidate looking at the screen or away"), plus face-presence, multi-face, and browser-event signals. This is ~80–90% reliable and runs free, in the browser.

We will frame the feature honestly as **"AI-assisted integrity flagging for human review"** — never "automatic cheating detection." False positives in an automated-reject system create legal and bias risk.

---

4. Phase A — Candidate video + true 1:1 interaction

Objective: the candidate's webcam is live in the interview session.

What changes

- **Frontend** (`web/src/features/interview/`): request **camera** permission

(today we only request mic), render the candidate's own video in a corner / side panel next to the avatar, and **publish the camera track to the LiveKit room** (LiveKit already carries our audio + avatar video, so the candidate video track is the same transport — no new infra).

- **Consent** (`data_gateway` consent ledger): a **new DPDP consent type

`video_capture`** must be granted before the camera turns on. This is our first candidate-side video, so it needs its own explicit consent record.

- **Storage decision (cost + DPDP):** by default **do NOT upload raw candidate

video** to our servers. Keep it client-side and ephemeral. Recording the full video is a separate, opt-in, consented feature (large storage + biometric data under DPDP §3(k)). For most use cases, only the *proctoring events* (Phase B) need to leave the browser, not the video itself.

Effort

~2–3 days (frontend + consent type + a migration for the new consent type).

Why this is first

Phase B (gaze) needs a camera stream to analyze. Phase A delivers the camera and the consent gate that Phase B builds on.

---

5. Phase B — Gaze tracking & proctoring pipeline

Objective: detect malpractice signals and produce a reviewable integrity score.

5.1 Detection — runs in the browser, not on our servers

Detection runs **client-side in a Web Worker** so it costs us no server compute and the raw video never leaves the candidate's machine (cheapest + safest for DPDP). The browser emits small **events**, not video frames.

Signal	Method	Reliability
Gaze direction / "looking away"	MediaPipe **FaceLandmarker** (head pose + eye landmarks)	~80–85%
Face present / absent	Same model (face count = 0)	~95%
Multiple faces in frame	Same model (face count > 1)	~98%
Tab / window switch, focus loss	Browser `visibilitychange` + `blur`	~99.9%
Copy / paste, fullscreen exit	Browser clipboard / fullscreen API	~99.9%
Second voice / background speech	Reuse existing **Silero VAD** in `interview_core`	~90%

Recommended model stack (all free, browser-based):

- **MediaPipe Tasks (FaceLandmarker)** — primary. 478 landmarks, head pose, eye region. ~30ms/frame, no calibration. `@mediapipe/tasks-vision`.
- **MediaPipe Iris** — optional add-on if we later want finer eye precision.
- **WebGazer.js** — only if we want true on-screen gaze coordinates (needs a ~30s 9-point calibration; more UX friction). Not needed for v1.
- A **Python / server-side** path (e.g. MediaPipe Python, OpenCV) is the **Tier-2 option **if we ever move detection server-side for tamper-resistance — but that adds GPU/CPU cost per concurrent session and re-introduces the DPDP video- transit problem.** Client-side first.**

> *Note on the CLAUDE.md "Python models" idea: gaze CAN be done with Python (MediaPipe Python), but doing it **server-side** means streaming each candidate's video to us — expensive at 10k concurrent and a DPDP liability. The browser (JS/WASM MediaPipe) is the right default; keep Python server-side detection as a Tier-2 tamper-resistance upgrade, gated on cost review.*

5.2 Event flow & scoring

- Web Worker runs inference every ~500ms.
- It emits lightweight events: `gaze_away_start/end`, `face_absent_start/end`, `multiple_faces`, `tab_blur/focus`, `second_voice`.
- Events are batched and POSTed to a new endpoint in `interview_core`.
- Events are stored against the session (new table — see 5.3).
- At session end, a **scorer** computes an **integrity score (0–100)** from time-in-flagged-states + event counts, plus a timeline.
- The score + timeline surface in the **admin dashboard** drill-in (`web/src/pages/admin/AdminInterviewDetail.tsx`) for **human review**.

5.3 New data model

```
integrity_events
id uuid pk
session_id uuid fk -> sessions(id) on delete cascade
event_type text -- gaze_away | face_absent | multiple_faces | tab_blur | second_voice | ...
started_at timestamptz
ended_at timestamptz null -- for ranged events
metadata jsonb -- e.g. {"confidence": 0.7, "yaw_deg": 35}

sessions (add columns)
integrity_score smallint null -- 0-100, computed at session end
```

```
proctoring_summary jsonb null -- counts + total flagged seconds
```

One Alembic migration in `data_gateway`. `interview_core` already maps a partial `sessions` model; add the two columns there too (read path).

5.4 Effort

~5–7 days: Web Worker + MediaPipe integration (2), event API + schema (1), integrity scoring (1), admin timeline UI (1–2), consent + tests (1).

5.5 Honesty / fairness guardrails (do not skip)

- Label everything "flag for review," never "cheating confirmed."
- Calibrate thresholds; webcam gaze is noisy (glasses, lighting, dark skin tones,

disability). Document known limitations.

- Make the candidate aware proctoring is active (consent + visible indicator).

5.6 Status — Phase B v1 SHIPPED (2026-06-19)

Built, tested, and live:

- Client-side detection (browser, on-device): **head-pose** gaze (3D rotation

matrix), **true eye-gaze** (iris blendshapes), **face-absent**, **multiple-faces**, plus **tab-switch / copy / paste / fullscreen-exit**. Raw video never leaves the device — only events.

- Ingestion API (`POST /api/sessions/{id}/integrity-events`, ownership-checked) +

`integrity_events` table + rolling `integrity_score` (0–100) + `proctoring_summary`.

- Pure decision logic extracted to `web/.../proctorLogic.ts` (debounce state

machine, warning selection, gaze decision) with **13 Vitest unit tests**; backend scoring + endpoint covered by **13 pytest tests**.

- **Live candidate warnings** (on-screen, ≥5s sustained) for face-absent /

looking-away / multiple-faces.

- Admin **integrity panel** with score, per-type flags, and a **time-ordered

event timeline**. Disclaimer: "AI-assisted flagging for human review."

- Defense-in-depth: per-event duration clamp (900s) so a client clock bug can't

blow up the score.

Deferred (by design — NOT in v1):

- `second_voice` **detection** — robustly distinguishing a second/background

speaker needs speaker **diarization** (or careful echo handling so the avatar's own audio isn't misread). Shipping a false-positive-prone version would undermine the "advisory" stance, so it's deferred. The scorer already reserves a weight for it (forward-compatible). Note: `multiple_faces` already covers the *visible* helper case.

- **Identity verification** (face snapshot + match vs ID photo + liveness) —

the plan's proctoring "Phase 2".

- **Web Worker** for detection (currently main-thread at ~2 fps) — scaling polish.
- **Self-hosting the MediaPipe model** (currently loads from Google's CDN).
- **Threshold calibration / optional candidate calibration step** — current

thresholds are sensible defaults, tunable in one place (`proctorLogic.ts` / `useProctoring.ts`).

- **Proctoring data retention policy** — fold `integrity_events` into the DPDP

retention/erasure job before production.

---

6. Phase C — Bulk hiring engine

Objective: support a company (or many) interviewing thousands of candidates in a hiring drive.

> ***Architectural reality check (decided 2026-06-16):*** *the chosen direction is* ***avatar-everywhere + fully-synchronous live*** *interviews. That is the most demanding and most expensive mode. At thousands of concurrent live avatar sessions we are bound by* ***Tavus/Simli vendor concurrency limits*** *(needs an enterprise quota, not the demo tier) and very likely exceed the* ***■12/session cap*** *in* `Final_stack.md`*. This must be validated by* `cfo-cost-watcher` *+ a real load test (the unused Locust scaffold, backlog B-029)* ***before*** *any government bid. Treat full-sync-avatar as the premium tier; consider voice-only or async for true mass screening if economics demand it.*

6.1 New domain model — campaigns & batches

The current model is 1:1 (one user, one ad-hoc session). Bulk hiring needs a campaign concept owned by a company/recruiter:

```
companies
id, name, ... -- the hiring org (or college)

campaigns
id, company_id, job_id, name
window_start, window_end -- when candidates may interview
settings jsonb -- avatar on/off, proctoring level, language, question count
status -- draft | open | closed

campaign_invites
id, campaign_id, candidate_email
token -- unique magic-link token
status -- invited | started | completed | expired
session_id uuid null fk -> sessions

sessions (add column)
campaign_id uuid null fk -> campaigns(id) -- null = self-serve / off-campus
```

6.2 New infrastructure — a real job queue

We have **no job queue today** (everything is synchronous FastAPI). Bulk needs one:

- **Recommended:** Arq or Celery on the **Upstash Redis** we already run.
- Queued jobs: send invites + reminder emails (Resend is in-stack), end-of-session

scoring, PDF generation, post-campaign report aggregation.

6.3 Worker pool & autoscaling

- The `interview_worker` (LiveKit agent) becomes a **horizontally-scaled pool**.

Each live interview = one agent process. The pool size is capped by the avatar vendor concurrency quota.

- On Tier-2 (AWS EKS Mumbai), this is an HPA-scaled deployment; on the demo tier

(Railway), it's manual replica scaling.

6.4 Recruiter console (new frontend area)

- CSV / ATS upload of candidate lists → generates `campaign_invites`.
- Live campaign dashboard: X invited / Y started / Z completed / avg score.
- Bulk export of scorecards.

- Lives alongside `web/src/pages/admin/` (reuses the admin_ops analytics patterns).

6.5 Effort

~3–4 weeks (campaign model + migrations, job queue, invite flow + emails, recruiter console, worker autoscaling, load test).

---

7. Intake scenarios (one engine, several front doors)

All scenarios drive the **same** interview engine; they differ only in how a session is created and what context is attached.

#	Scenario	How a session is created	Notes
1	**Off-campus individual**	Candidate self-registers, picks a role, starts	Today's flow. `campaign_id = null`.
2	**Resume-driven**	Candidate uploads resume → questions grounded in it	Already wired (`resume_text` → interviewer prompt). Future: resume↔JD fit score as a pre-gate.
3	**Bulk / campaign**	Recruiter invites via campaign; candidate clicks magic link	Phase C. `campaign_id` set; settings inherited from campaign.
4	**Admin-managed**	Admin creates jobs/JDs, configures campaigns, reviews results	`admin_ops` + recruiter console; admin role (see `grant_admin.py`).

The key design principle: **scenario = a thin layer over session creation**, not a fork of the interview engine. This keeps one codebase for all markets.

---

8. Production security (required before real PII at scale)

Maps to DPDP Act 2023 + the RFP NFRs in `HLD.md`. Several items already exist; this lists what must be true for production.

Already in place

- Pluggable auth, JWT with issuer/audience validation, role-gated admin.
- DPDP consent ledger, right-to-erasure pipeline, audit log, soft-delete + retention.
- Secrets via env / Pydantic settings (no hardcoded secrets).

Must add / harden for production

- **Biometric data governance (NEW, critical):** camera + gaze data are

biometric under DPDP §3(k). Needs: explicit `video_capture` / `proctoring` consent types, a defined retention window, encryption at rest if any video is ever stored, and inclusion in the erasure pipeline.

- **India data residency:** Tier-2 production = AWS Mumbai only (already the

plan). Tavus is US-hosted and demo-only — must be replaced by the custom RPM avatar (Tier-2) before a government deployment.

- **Rate limiting & abuse controls:** invite-token brute-force protection,

per-IP and per-account limits (login limiter exists; extend to invites/sessions).

- **Transport & secrets:** TLS everywhere, secrets in a manager (AWS Secrets

Manager on Tier-2), rotate the demo keys before production.

• **Proctoring fairness & transparency:** documented accuracy limits, human-in-the-loop review, candidate notification — to defend against bias/legal claims.

• **Tenant isolation (multi-company):** campaign/company data must be scoped so one company can never read another's candidates or results.

• **Sign-off gates:** `security-auditor` review before any production deploy; `cfo-cost-watcher` review of per-session cost before any L1 bid.

---

9. Recommended sequencing

Order	Phase	Why	Rough effort
1	**A — Candidate video**	Unlocks B; small, high-impact	2–3 days
2	**B — Gaze proctoring**	Biggest differentiator; self-contained	5–7 days
3	**C — Bulk engine**	The scale story; largest build	3–4 weeks
4	**Security hardening**	Continuous; gate before production	ongoing

Phases A + B can ship on the current architecture without the bulk engine, so the proctoring story is demoable long before the scale work lands.

---

10. Open decisions to confirm before building

• **Record candidate video, or events-only?** (Events-only = cheaper + DPDP-safe; recording = stronger evidence but storage + biometric burden.)

• **Bulk mode economics:** confirm avatar-everywhere full-sync is financially viable at scale, or define a voice-only/async fallback for mass screening.

• **Proctoring strictness:** advisory flags only (recommended) vs. hard auto-actions (not recommended for v1).

• **Multi-company now or single-tenant first?** (Affects whether we build the `companies` table + tenant isolation in Phase C or defer it.)

End of plan. Update this document as decisions are made; it is the source of truth for the expansion scope.